

Materi Komprehensif: **Arsitektur RTOS dan Implementasi** **Peripheral Interfacing**

Muhammad Dzakey Athallah

NIM: 40040324620011

Kelas: TRO A

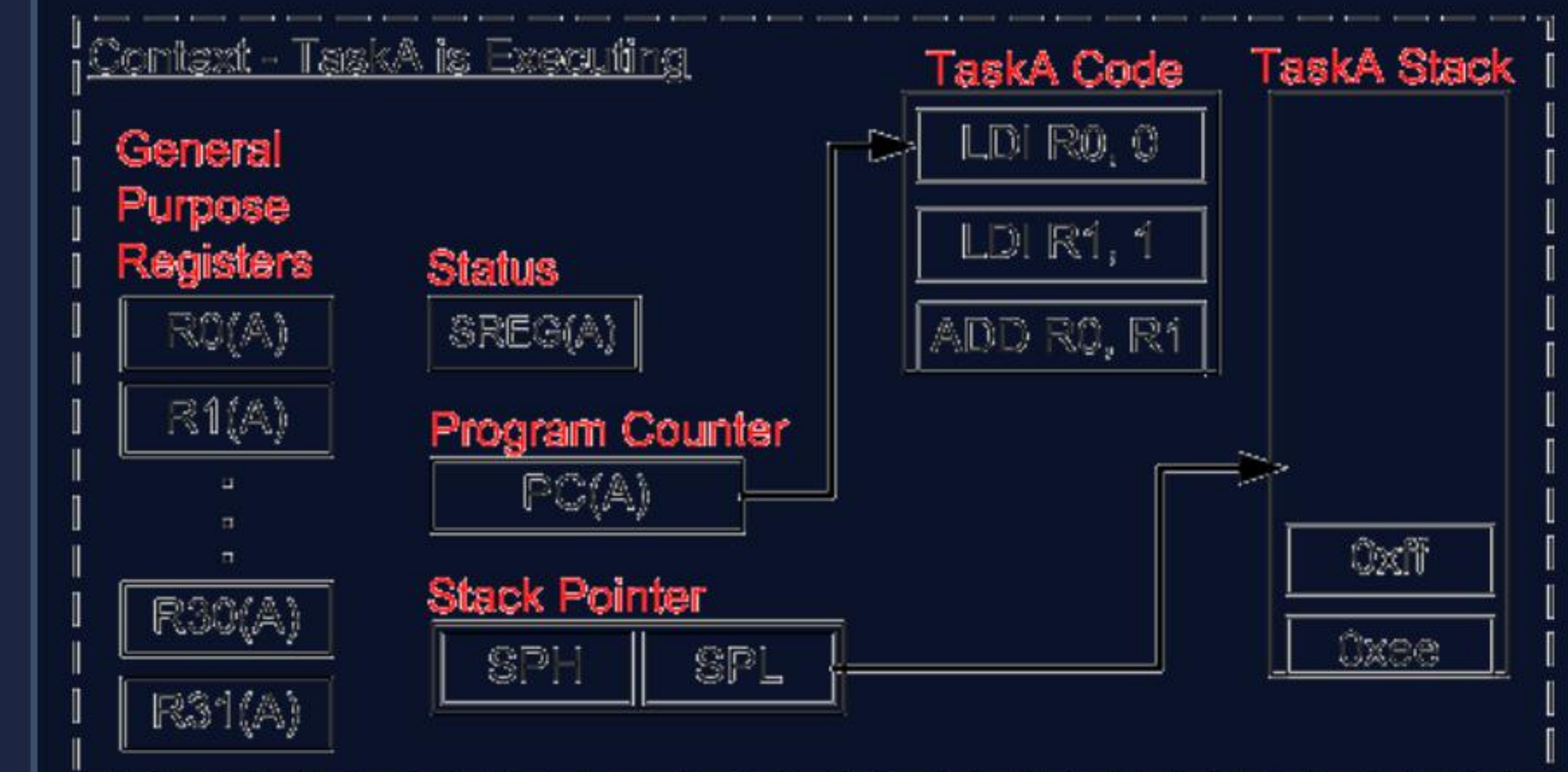
1. Konsep Inti Task dan Eksekusi Konkuren

Dalam pendekatan bare-metal (tanpa OS), program berjalan secara sekuensial dalam satu Super Loop (misalnya `void loop()` pada Arduino). Dalam RTOS, paradigma ini berubah menjadi eksekusi konkuren.

Task sebagai Unit Eksekusi: Sebuah Task (atau thread) adalah blok kode mandiri (biasanya berupa infinite loop) yang memiliki tujuannya sendiri (misal: Task A membaca sensor, Task B mengontrol motor, Task C menangani WiFi). RTOS memberikan ilusi bahwa setiap task memiliki CPU-nya sendiri.

Context Switch & Stack Management: Karena CPU sebenarnya hanya satu (pada single-core MCU), kernel RTOS harus memindah-mindahkan eksekusi antar task dengan sangat cepat. Proses ini disebut Context Switch.

- Saat beralih, status register CPU saat itu disimpan ke dalam ruang memori khusus milik task tersebut yang disebut Stack.
- Setiap task membutuhkan alokasi RAM yang presisi untuk Stack-nya (agar tidak terjadi Stack Overflow) dan struktur data kontrol yang disebut Task Control Block (TCB).

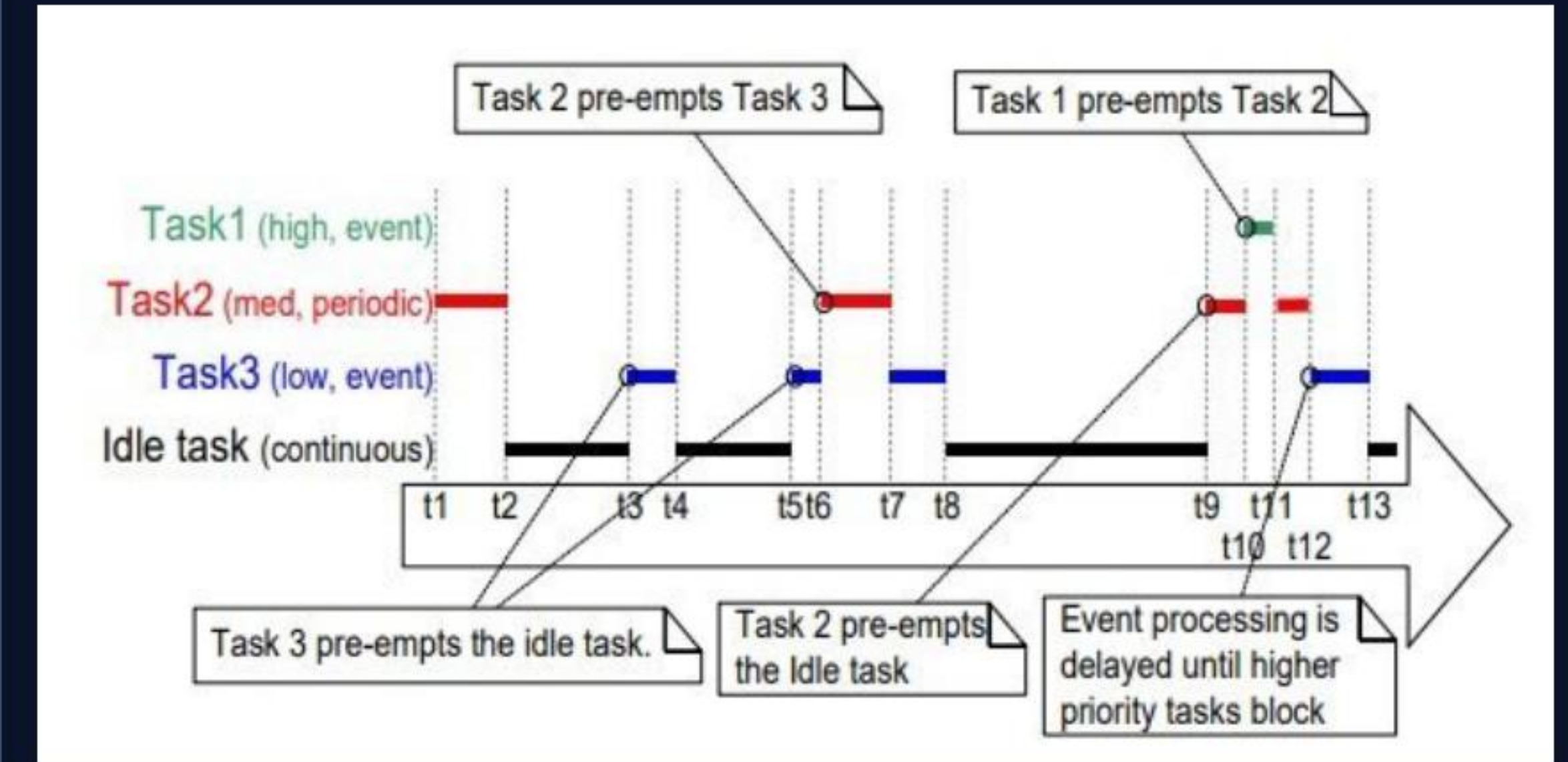


2. Penjadwalan (Scheduling) Berbasis Prioritas

RTOS seperti FreeRTOS menggunakan algoritma Preemptive Priority-Based Scheduling.

Priority: Setiap task diberi angka prioritas. Kernel akan selalu mengeksekusi task dengan prioritas tertinggi yang siap (Ready) berjalan.

Preemptive: Jika sebuah task prioritas tinggi tiba-tiba siap berjalan (misal karena sebuah interupsi atau delay telah selesai), ia akan langsung "merebut" (preempt) CPU dari task prioritas rendah yang sedang berjalan saat itu juga, tanpa harus menunggu task tersebut selesai.



3. Sinkronisasi dan Manajemen Interupsi (ISR)

Penanganan Hardware Interrupt (ISR) di RTOS memiliki aturan yang jauh lebih ketat dibandingkan bare-metal.

ISR-safe API (FromISR)

Di dalam fungsi interupsi, eksekusi harus secepat mungkin dan tidak boleh ada fungsi yang bersifat blocking (seperti delay atau menunggu koneksi). Oleh karena itu, RTOS menyediakan fungsi khusus yang diakhiri dengan FromISR (misal: `xQueueSendFromISR()`) agar aman dipanggil dari dalam interupsi tanpa merusak scheduler.

Binary Semaphore (ISR-to-Task Signaling)

Cara terbaik menangani interupsi berat adalah dengan teknik Deferred Interrupt Processing.

ISR perangkat keras dipicu, ia hanya melakukan tugas ultra-cepat (misal membersihkan flag interupsi), lalu memberikan sinyal (Give Semaphore).

Sebuah Handler Task berprioritas sangat tinggi yang sedang tertidur (Block state) akan terbangun (Take Semaphore) dan melakukan pemrosesan data yang berat.

4. Inter-Task Communication & Shared Resources (1/2)

Ketika banyak task berjalan, mereka harus bisa berbagi data dan perangkat keras tanpa saling merusak (race condition).

Queue (Antrean)

Mekanisme antrean berbasis FIFO (First In, First Out) untuk mengirim data antar task atau dari ISR ke Task. Misalnya, Task Pembaca Serial mem-parsing data, lalu memasukkannya ke Queue. Task Kontrol Motor akan membaca dari Queue tersebut dan mengeksekusinya secara berurutan.

Mutex (Mutual Exclusion)

Digunakan untuk memproteksi Shared Resource (sumber daya yang digunakan bersama), seperti jalur I2C, SPI, atau layar LCD. Jika Task A sedang memakai I2C (memegang Mutex), Task B yang ingin memakai I2C harus mengantre (masuk status Blocked) sampai Task A selesai dan melepaskan Mutex.

4. Inter-Task Communication & Shared Resources (2/2)

Priority Inheritance: Fitur canggih pada Mutex. Jika Task Prioritas Rendah memegang Mutex, dan Task Prioritas Tinggi membutuhkannya, sistem akan sementara menaikkan prioritas Task Rendah tersebut agar ia cepat selesai dan segera melepaskan Mutex. Ini mencegah masalah Priority Inversion (kemacetan sistem).

Event Group: Bendera status sinkronisasi. Memungkinkan sebuah Task menunggu kombinasi dari beberapa event terjadi (menggunakan logika AND / OR). Misal: Task Sistem Siap baru berjalan JIKA (Sensor Calibrated AND Network Connected AND Motor Homing Done).

 FreeRTOS Queue mechanism for intertask communication

5. Implementasi Peripheral Perangkat Keras (1/2)

Penerapan RTOS secara masif mengubah cara kita memprogram sensor dan aktuator.

GPIO & vTaskDelay(): Blinking LED atau logika I/O sederhana tidak boleh menggunakan delay() biasa karena memblokir CPU (wasting cycles). Menggunakan vTaskDelay() membuat task tersebut berstatus Blocked, sehingga CPU 100% bisa digunakan oleh task lain selama masa tunggu tersebut.

External Interrupt & Encoder 2-Pin: Sinyal fase A/B dari rotary encoder dihubungkan ke pin External Interrupt. Saat motor berputar cepat, ISR akan terus-menerus terpicu untuk menghitung posisi pulsa, lalu nilai kalkulasi posisinya dikirim ke RTOS Queue agar PID Controller Task dapat menghitung kecepatan (RPM) secara presisi.

Serial Communication (UART): Transmisi data serial sangat lambat dibanding kecepatan CPU. Sistem menggunakan interupsi RX untuk menangkap setiap byte yang masuk dan menaruhnya di Queue. TX juga bisa menggunakan Queue atau interupsi buffer empty, sehingga CPU tidak pernah nganggur menunggu pengiriman karakter serial.

5. Implementasi Peripheral Perangkat Keras (2/2)

> Analog IO (ADC dan DAC):

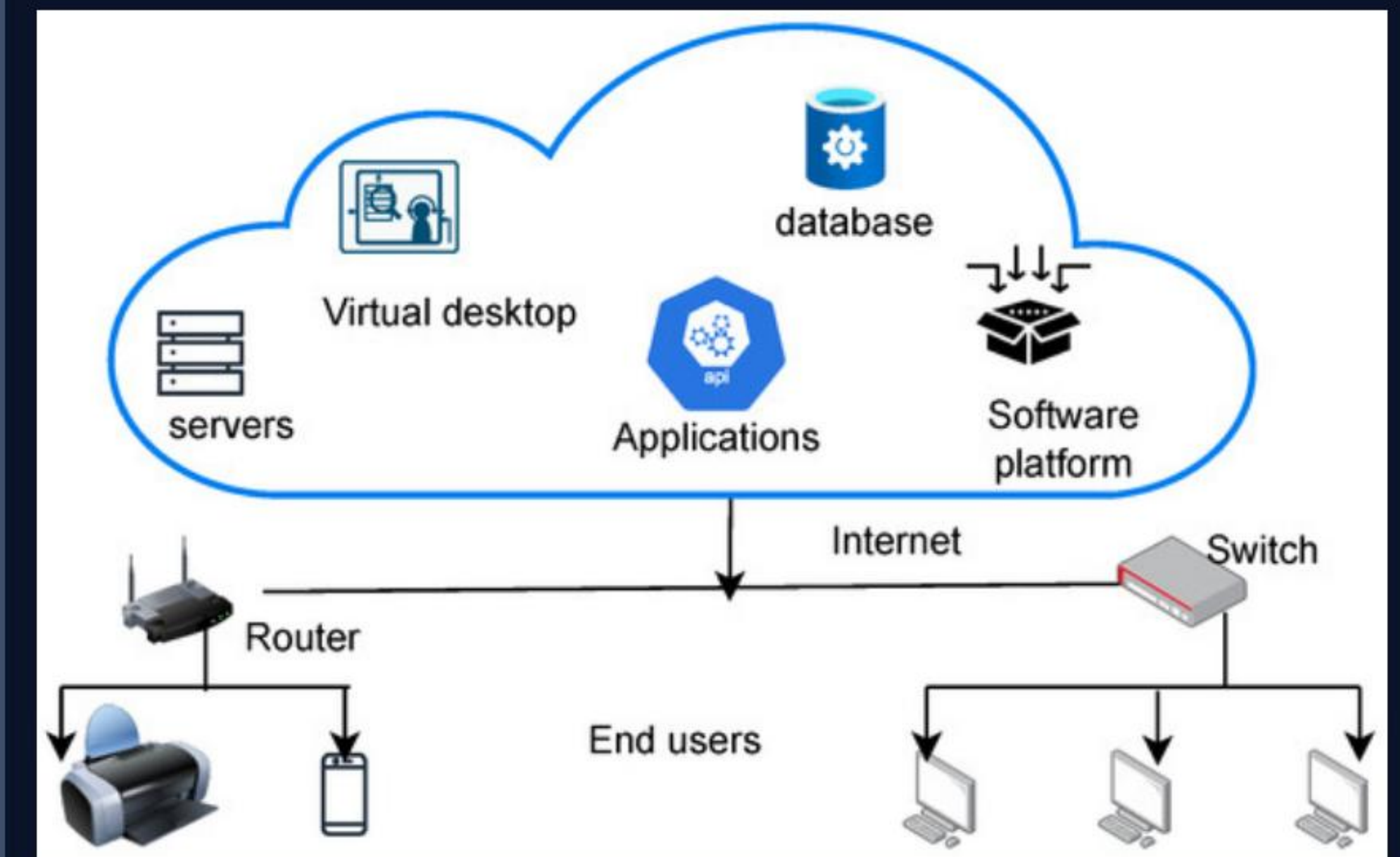
- ADC Task: Berjalan periodik (misal setiap 10ms) untuk mengambil sampel tegangan sensor. Jika menggunakan DMA, interupsi DMA (transfer complete) akan membangunkan task pemroses data lewat Semaphore.
- DAC Task: Menerima data dari Queue perhitungan algoritma (seperti output PID) dan mengubahnya menjadi sinyal tegangan analog keluaran.

> Shared Bus (I2C dan SPI): Sering kali kita memiliki banyak sensor I2C (seperti MPU6050, OLED, RTC) di jalur SDA/SCL yang sama, namun dibaca oleh task yang berbeda-beda. Pemanggilan fungsi I2C_Read() atau SPI_Transmit() mutlak harus dibungkus dengan xSemaphoreTake(Mutex) dan xSemaphoreGive(Mutex) agar transmisi satu slave tidak disela di tengah jalan oleh transmisi ke slave lain.

6. Komunikasi Antar-MCU (Distributed Systems)

Dalam sistem terdistribusi, arsitektur RTOS sangat ideal untuk mengelola paket data yang masuk secara asinkron dari node (MCU) lain.

Sebuah Communication Task khusus dapat didedikasikan untuk mendengarkan jalur bus (UART/CAN Bus/RS485/SPI Master-Slave), memvalidasi paket data (seperti Checksum/CRC), lalu mendistribusikan command dari MCU lain tersebut ke berbagai task lokal melalui antrean pesan (Message Queues).



Q&A

Terima kasih atas perhatiannya.

Muhammad Dzakey Athallah

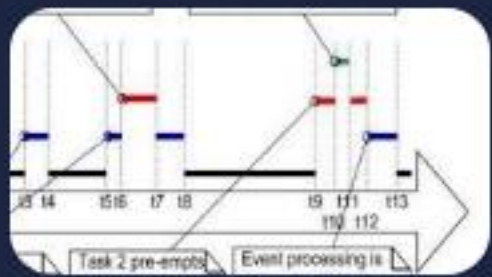
40040324620011 | Kelas TRO A

Image Sources



<https://www.freertos.org/media/2018/AtoB1.gif>

Source: www.freertos.org



<http://microcontrollerslab.com/wp-content/uploads/2020/06/FreeRTOS-Preemptive-scheduling-algorithm-example.jpg>

Source: microcontrollerslab.com



<https://i.ytimg.com/vi/GnMoveG28Cc/maxresdefault.jpg>

Source: www.youtube.com



https://www.mdpi.com/computers/computers-12-00198/article_deploy/html/images/computers-12-00198-g005-550.jpg

Source: www.mdpi.com